

I analyzed the uploaded Zoom notes. This session is essentially the **first conceptual lecture of the RAG (Retrieval-Augmented Generation) module**. The instructor repeatedly emphasizes understanding the **fundamentals and architecture** before writing any code.

Below are the notes reorganized as they would appear in a GenAI textbook rather than a Zoom transcript.

Retrieval-Augmented Generation (RAG)

Expert Notes (Structured Learning)

Module Overview

This chapter introduces one of the most important concepts in modern Generative AI.

Unlike Fine-tuning, RAG does **not train the LLM**.

Instead, it provides the LLM with **relevant external knowledge** before asking it to generate an answer.

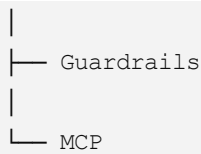
Where RAG Fits in GenAI

According to the course flow:

Generative AI



```
|
|— Encoding
|
|— Embeddings
|
|— Transformer Architecture
|
|— LLM APIs
|
|— Fine-tuning
|
|— RAG ← Current Module
|
|— Agentic AI
|
|— Prompt Engineering
|
|— Evaluation
```



The instructor mentions spending approximately:

- 3 months on foundations
- 8–10 classes on Fine-tuning
- 6–8 classes on RAG
- Followed by Agentic AI

Why RAG is Important

The instructor clearly mentions:

Fine-tuning is NOT the most common work in industry.

Most real-world GenAI applications involve

- Chatbots
- Enterprise Search
- Knowledge Assistants
- Company Q&A
- Document Search

These are almost always built using RAG.

Definition of RAG

RAG stands for

Retrieval
Augmentation
Generation



Retrieval

Retrieve relevant information from external data.

Examples:

- PDFs

- Word documents
 - Websites
 - Databases
 - Internal company documents
 - Knowledge Bases
-

Augmentation

Augmentation means

Enrich the user's prompt by adding retrieved context.

Instead of

User Question



the model receives

Context



+

User Question

+

Instructions

Generation

The LLM generates the answer using

- User question
 - Retrieved context
 - Prompt instructions
-

One-Line Interview Definition

RAG is a technique that retrieves relevant external knowledge and provides it as context to an LLM before generating the final response.

Core Philosophy of RAG

The instructor repeatedly emphasizes:

"LLM doesn't know everything."

Therefore,

Instead of retraining the model

↓

Give the model the required information
before asking the question.



Classical RAG Pipeline

DATA SOURCES

|

Data Parsing

|

Chunking

|

Embedding

|

Vector Database

|

User Question

|

Embedding

|

Similarity Search



|

Relevant Chunks Retrieved

|

Context

+

Question

+

Instructions

|

LLM

|

Generated Answer

Two Major Parts of RAG

The instructor divides the entire pipeline into two sections.

Part 1

Ingestion & Indexing

Responsible for preparing knowledge.

Includes

- Reading documents
- Parsing
- Chunking
- Embeddings
- Vector Database

Part 2

Retrieval & Generation

Responsible for answering questions.

Includes

- Similarity Search
 - Retrieving Context
 - Prompt Augmentation
 - LLM Response
-

Three Major Components

The instructor simplifies RAG into three components.

1. Data Ingestion

Responsible for creating the knowledge base.

Includes

- Data Parsing
 - Chunking
 - Embeddings
 - Vector Database
-

2. Retrieval

Responsible for finding the correct information.

Main task:

User Query



↓

Embedding

↓

Similarity Search

↓

Relevant Chunks

3. Generation

LLM generates the answer using

Prompt



+

Retrieved Context

+

User Query

Difficulty Level

According to the instructor:

Difficult Areas

Data Parsing

Reasons:

- Different document formats
- Tables
- Images
- OCR
- Metadata
- Layout understanding

Retrieval

Reasons

Need to retrieve

the correct chunk

not

just any chunk.

Retrieval quality determines

the final answer quality.

Easier Part

Generation

Because

modern LLMs already possess excellent reasoning capability.

The main work is prompt design.

What is Data Ingestion?

Pipeline:

Raw Documents



↓

Parsing

↓

Chunking

↓

Embeddings

↓

Vector Database

What is Retrieval?

User Question



↓

Embedding

↓

Similarity Search

↓

Top Relevant Chunks

What is Augmentation?

Question



+

Retrieved Context

+

Instructions

↓

LLM

What is Generation?

LLM



↓

Grounded Response

Example

User asks

What is our refund policy?



LLM alone

May hallucinate.



With RAG

Retrieve from company policy

Refunds allowed within 7 days.



Prompt becomes

Context



Refunds allowed within 7 days.

Question

What is our refund policy?

LLM now produces

Refunds are allowed within 7 days.



Why RAG is Needed

The instructor explains four major reasons.

1. Private Data

LLMs are NOT trained on

- Company documents
- Internal databases
- Enterprise knowledge

RAG connects the LLM with that information.

2. Latest Information

LLMs have knowledge cutoffs.

Example

Current temperature

Current company policies

Latest products

Latest regulations

Need retrieval.

3. Hallucination Reduction

Hallucination

=

LLM confidently gives the wrong answer.

Example

LLM

Refund period = 30 days



Company document

Refund period = 7 days



Using RAG

The answer is grounded in retrieved documents.

4. Avoid Continuous Fine-tuning

If documents change

Every day

Every week

Every month

Fine-tuning becomes expensive.

Instead

Simply update

Documents



↓

No retraining required.

RAG vs Fine-tuning

RAG	Fine-tuning
Adds external knowledge	Changes model behaviour
No weight updates	Model weights change
Lower cost	Higher cost
Easy document updates	Retraining required
Best for enterprise search	Best for behaviour/style customization

Chunking

Definition

Breaking a large document into smaller meaningful pieces.

Instructor notes

Chunking is sometimes optional depending on use case.

Embeddings

Convert text into vectors.

Both

Documents

and

User Queries

are converted into embeddings.

Similarity search happens on embeddings.

Vector Database

Purpose

Store embeddings.

Examples (to be covered later)

- Chroma
 - FAISS
 - Pinecone
 - Milvus
 - Weaviate
-

Similarity Search

Process

Query Embedding



↓

Compare with Document Embeddings

↓

Retrieve nearest vectors

↓

Return relevant chunks

Data Sources

The instructor mentions RAG can retrieve from

- PDFs
- DOCX
- CSV
- JSON

- HTML
- SQL
- Internal Databases
- Company Portal
- Websites
- Cloud Storage
- Images
- Audio
- Video

Data Modalities

Nature of data

Text

Images

Audio

Video



Most RAG systems initially focus on

- Text
- Images

File Formats

Examples

PDF

DOCX

CSV

JSON

YAML

HTML

MP3



Data Structures

The instructor classifies data as

Structured

- SQL
- CSV
- Excel

Semi-structured

- JSON
- YAML
- HTML

Unstructured

- PDF
 - Images
 - Audio
 - Video
-

LangChain vs LangGraph

Instructor recommendation

Focus only on

- LangChain
- LangGraph

Avoid trying many frameworks simultaneously.

Reason

Concepts matter more than frameworks.

Framework Usage

LangChain

Used for

- Classical RAG
 - Document loading
 - Embeddings
 - Retrieval
 - Prompt chains
-

LangGraph

Used for

- Agentic RAG
 - Multi-agent workflows
 - Orchestration
 - Memory
 - State management
-

Future Topics Mentioned

The instructor plans to cover

- Document AI
 - OCR
 - Data Parsing
 - Metadata
 - Chunking Strategies
 - Vector Databases
 - Similarity Search
 - Retrieval Pipelines
 - Prompt Engineering
 - Advanced RAG
 - Memory
 - Evaluation
 - Guardrails
 - Vectorless RAG
 - Agentic RAG
 - MCP Integration
-

Vectorless RAG

The instructor briefly introduces Vectorless RAG.

Instead of

Embedding



↓

Vector Database

It retrieves directly from documents using traditional retrieval methods such as keyword-based search (e.g., BM25).

Important takeaway:

- Vectorless RAG is **not a replacement** for vector-based RAG.
- It may work well for some use cases but should be evaluated based on retrieval quality.

Practical Learning Roadmap (Instructor Plan)

RAG



```
|
├─ Introduction
|
├─ Why RAG?
|
├─ RAG Pipeline
|
├─ RAG vs Fine-tuning
|
├─ Data Parsing (Document AI)
|
├─ Chunking
|
├─ Embeddings
|
├─ Vector Databases
|
├─ Retrieval
|
├─ Similarity Search
|
└─ Prompt Engineering
```

—	Generation
—	Advanced RAG
—	Agentic RAG
—	MCP Integration

Key Takeaways

- 1. RAG is a pipeline, not an LLM or a model.**
- 2. The three pillars of RAG are Data Ingestion, Retrieval, and Generation.**
- 3. The most challenging production problems are data parsing and accurate retrieval; generation is comparatively easier due to strong modern LLM reasoning.**
- 4. RAG is preferred over fine-tuning for enterprise knowledge, frequently changing data, and private documents.**
- 5. Mastering concepts such as embeddings, vector databases, similarity search, and prompt augmentation is more valuable than learning many frameworks.**
- 6. LangChain is the primary framework for classical RAG, while LangGraph becomes important for Agentic RAG and orchestration.**

These notes preserve the instructor's explanations but reorganize them into a structured, reference-style guide suitable for revision or interview preparation.